

Lab 2: Serial Communication & Bar Graph

Engineer: Stanley Etienne / J00974053

Course: ECEL-360

Date: 01/28/2026

Objective

This lab's goal was to use the ESP32 microcontroller to build and implement a serial-controlled LED bar graph. The system allows the user to input a numeric value through the Serial Monitor, which is then processed by the ESP32 to illuminate a proportional number of LEDs. This lab demonstrates how serial communication can be used to control numerous digital outputs and see numerical data in real time.

Materials

- 1× ESP32 microcontroller (Freenove ESP32 WROVER)
- 1× Breadboard
- 10× LEDs
- 10× 220 Ω resistors
- Jumper wires
- USB cable
- Computer with Arduino IDE

Background and Theory

A popular technique for data transfer between a microcontroller and a computer is serial communication. Software-based control of hardware outputs is made possible by the Arduino IDE's Serial Monitor, which allows users to send commands or numerical values straight to the ESP32.

An LED bar graph uses many LEDs to visually display numerical data. Each LED reflects a fraction of the whole input range. The system can graphically represent magnitude or level by splitting the input range into equal parts.

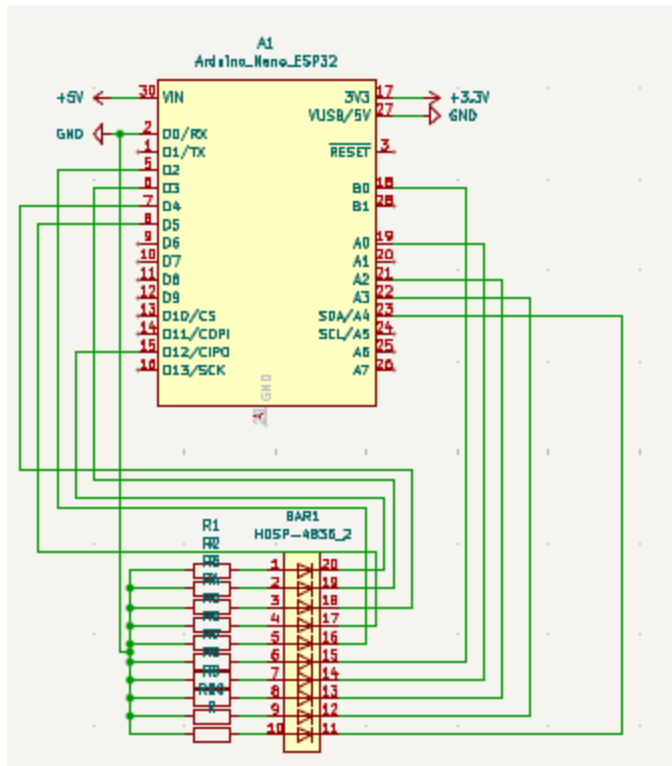
In this lab, the ESP32 gets a number between 0 and 100 via serial input. Each LED

represents 10% of the entire input range after this value has been scaled and mapped to a range of 0 to 10 LEDs. As the input value grows, additional LEDs are lit, generating a bar graph pattern. This illustrates how an embedded system can integrate serial input, data parsing, and array-based GPIO control.

Schematic and Procedure

Wiring Configuration

- GPIO 15 → 220 Ω resistor → LED 1 anode
- GPIO 4 → 220 Ω resistor → LED 2 anode
- GPIO 0 → 220 Ω resistor → LED 3 anode
- GPIO 2 → 220 Ω resistor → LED 4 anode
- GPIO 5 → 220 Ω resistor → LED 5 anode
- GPIO 18 → 220 Ω resistor → LED 6 anode
- GPIO 19 → 220 Ω resistor → LED 7 anode
- GPIO 21 → 220 Ω resistor → LED 8 anode
- GPIO 22 → 220 Ω resistor → LED 9 anode
- GPIO 23 → 220 Ω resistor → LED 10 anode
- All LED cathodes → GND
- ESP32 GND → Common ground rail



(Similar board using the same principle)

Procedure

1. The ESP32 was connected to the breadboard and ten LEDs were wired to the specified GPIO pins using current-limiting resistors.
2. The Arduino IDE was configured for the Freenove ESP32 WROVER board and the correct serial port.
3. All LED pins were initialized as digital outputs and set to a LOW state.
4. The Serial Monitor was opened at 115200 baud with the line ending set to "Newline."
5. A numeric value between 0 and 100 was entered into the Serial Monitor.
6. The ESP32 parsed the serial input and illuminated the corresponding number of LEDs to form a bar graph.

Code

```
// Number of LEDs used in the bar graph
const int NUM_LEDS = 10;
```

```

// GPIO pins connected to each LED (from left to right)
int ledPins[NUM_LEDS] = {15, 4, 0, 2, 5, 18, 19, 21, 22, 23};

// Stores how many LEDs should be turned ON
int ledsOn = 0;

void setup() {
    // Initialize serial communication at 115200 baud
    Serial.begin(115200);

    // Configure each LED pin as an output and ensure all LEDs start OFF
    for (int i = 0; i < NUM_LEDS; i++) {
        pinMode(ledPins[i], OUTPUT);
        digitalWrite(ledPins[i], LOW);
    }

    // Prompt user for input through the Serial Monitor
    Serial.println("Type a number (0-100) and press ENTER:");
}

void loop() {
    // Check if data is available in the Serial buffer
    if (Serial.available()) {

        // Read the full line of input until a newline character is received
        String input = Serial.readStringUntil('\n');

        // Convert the received string into an integer value
        int value = input.toInt();

        // Validate that the input is within the acceptable range
        if (value >= 0 && value <= 100) {

            // Determine how many LEDs should be ON (each LED represents 10%)
            ledsOn = value / 10;

            // Update LED bar graph based on the calculated value
            for (int i = 0; i < NUM_LEDS; i++) {
                // Turn ON LEDs below the threshold, turn OFF the rest
                digitalWrite(ledPins[i], (i < ledsOn) ? HIGH : LOW);
            }

            // Display feedback in the Serial Monitor
            Serial.print("Value: ");
            Serial.print(value);

```

```
    Serial.print(" → LEDs ON: ");  
    Serial.println(ledsOn);  
  }  
}  
}
```

Results

The serial input values were faithfully reflected in the LED bar graph. The right number of LEDs illuminated when values between 0 and 100 were entered. For instance, three LEDs turned on when the value was 30, and eight LEDs were illuminated when the value was 80. Stable and predictable operation was ensured by the system's consistent disregard of values outside the acceptable range.

Conclusion

This lab successfully used the ESP32 microcontroller to operate a 10-LED bar graph via serial communication. In order to offer a clear visual representation of numerical data, the system parsed user input and mapped it to various GPIO outputs. Important embedded systems principles, such as array-based output control, input validation, and serial data handling, were emphasized in this lab.